Phase 3 Technical Report: Model Implementation and Execution for Wearable Biosignal Stress Classification

Group ID: F26-02
Track A: Biosignal Processing and Affective Computing

October 20, 2026

Team Members

Name	Roll no
Muhammad Ahmad Khan	[24L-2541]
Muhammad Salman Ali	[24L-2542]
Rafia Mohsin	[24L-2595]

Abstract

This report details the implementation and execution phase (Phase 3) for the automated classification of acute cognitive stress using multimodal wearable biosignals from the WESAD benchmark dataset. Building upon the preprocessed feature matrix generated in Phase 2, two complementary architectures were engineered, trained, and validated: an ensemble Random Forest baseline via Scikit-Learn and a temporal 1D Convolutional Neural Network (1D-CNN) implemented in PyTorch. Both models were trained on 5,504 non-overlapping five-second statistical windows extracted from chest Electrocardiogram (ECG) and Electrodermal Activity (EDA) channels. Strict leak-free standardization and stratified cross-validation protocols were enforced. The Random Forest baseline achieved an initial classification accuracy of 98.73% (F_1-score: 0.98), while the 1D-CNN achieved an accuracy of 96.37% (F_1-score: 0.95). Implementation challenges, architecture justifications, and empirical performance metrics are documented.

1 Introduction and Objectives

Continuous stress monitoring using wearable physiological telemetry requires algorithms capable of processing high-frequency, noisy data while maintaining low computational latency. The primary objective of Phase 3 is to construct, train, and benchmark two distinct algorithmic paradigms on the preprocessed WESAD dataset:

1. A classical, highly interpretable Machine Learning (ML) baseline utilizing an ensemble decision forest.
2. A Deep Learning (DL) architecture capable of localized feature abstraction across biosignal modalities using 1D temporal convolutions.

2 Model Selection and Justification

2.1 Random Forest Classifier (Scikit-Learn)

The Random Forest ensemble was chosen as the baseline for tabular biomedical features based on the following characteristics:

- **Non-linear Boundary Delineation:** Physiological responses to acute stress exhibit non-linear interactions across modalities. Decision trees naturally segment feature space without imposing Gaussian or linear assumptions.
- **Colinearity Invariance:** Statistical features derived from the same sensor (e.g., EDA_{Mean}, EDA_{Max}, and EDA_{Min}) exhibit extreme pairwise colinearity. Random sub-sampling of feature subsets at each split prevents dominant features from masking subtle cardiac indicators.
- **Variance Reduction via Bagging:** By aggregating predictions across 100 decorrelated decision trees, the model mitigates the high variance inherent to biological time-series data.

2.2 1D Convolutional Neural Network (PyTorch)

A custom 1D-CNN was implemented to explore deep feature extraction:

- **Cross-Channel Inter-dependency:** By treating the 8 extracted statistical metrics as a pseudo-sequential array with depth, 1D convolution kernels ($k = 3$) capture local interactions between adjacent cardiac and electrodermal statistics.
- **Parameter Efficiency:** Compared to fully connected multi-layer perceptrons, weight sharing in 1D convolutional layers reduces parameter counts, mitigating catastrophic overfitting on small-to-medium sample spaces.

3 System Architecture and Implementation

3.1 Software and Framework Stack

The complete pipeline was developed using Python in a containerized GPU-accelerated environment:

- **Data Manipulation:** NumPy (v1.24) and Pandas (v2.0) for vector indexing and structured DataFrame manipulation.
- **Preprocessing and Classical ML:** Scikit-Learn (v1.3) for data partitioning, leak-free feature scaling, and ensemble modeling.
- **Deep Learning Engine:** PyTorch (v2.1) utilizing CUDA tensors for forward-backward gradient propagation, loss tracking, and GPU inference.
- **Evaluation and Visualization:** Matplotlib and Seaborn for confusion matrix generation and metric compilation.

3.2 Network Architecture Details

The 1D-CNN architecture (`StressCNN1D`) was structured with the following layers:

$$X \in \mathbb{R}^{B\times 1\times 8} \xrightarrow{\text{Conv1D}_{16}} \text{BN} \xrightarrow{\text{ReLU}} \xrightarrow{\text{Conv1D}_{32}} \text{BN} \xrightarrow{\text{ReLU}} \xrightarrow{\text{AvgPool}} \xrightarrow{\text{Dropout}_{0.2}} \xrightarrow{\text{Linear}} \hat{y} \in \mathbb{R}^{B\times 2} \quad (1)$$

Batch Normalization (BN) is applied after each convolution to stabilize gradient descent across varying physiological amplitudes, followed by an Adaptive Average Pooling layer collapsing the sequence dimension prior to dense classification.

4 Experimental Setup and Training Pipeline

4.1 Data Ingestion and Leakage Prevention

The input feature table (`WESAD_ML_Ready_Features.csv`) contains 5,504 rows, where each row represents a 5-second non-overlapping temporal window extracted at 700 Hz across 15 subjects (S2–S17, excluding S1 and S12).

To ensure strict methodological rigor and prevent data leakage:

1. The dataset was partitioned *first* into an 80% training set ($N_{\text{train}} = 4,403$) and a 20% test set ($N_{\text{test}} = 1,101$) with stratification on the target state label (`Relaxed` vs. `Stressed`).
2. A Z-score standardizer (`StandardScaler`) was fitted *strictly* on the training partition:

$$z = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}} \tag{2}$$

3. The test partition was subsequently transformed using μ_{train} and σ_{train} without recalculation.

4.2 Hyperparameter Configuration

Table 1 outlines the operational hyperparameters selected for both models.

Table 1: Model Hyperparameter Configurations

Hyperparameter	**Random Forest Baseline**	**1D-CNN (PyTorch)**
Estimators / Epochs	$n_{\text{estimators}} = 100$	20 Epochs
Optimization Algorithm	Gini Impurity Splitting	Adam ($\beta_1 = 0.9, \beta_2 = 0.999$)
Learning Rate (α)	N/A	1×10^{-3}
Weight Decay	N/A	1×10^{-4} (L_2 regularization)
Batch Size	Full Dataset	32 samples per mini-batch
Loss Function	Cross-Entropy Splitting Criterion	Categorical Cross-Entropy Loss
Regularization	max_depth $= 10$, min_samples_split $= 4$	Dropout ($p = 0.20$) + BatchNorm

5 Experimental Results and Discussion

5.1 Comparative Quantitative Performance

Table 2 summarizes the test set classification performance across both models on the 1,101 held-out evaluation windows.

Table 2: Test Partition Classification Metrics ($N_{\text{test}} = 1,101$)

Model	**Accuracy**	**Class**	**Precision**	**Recall**	**F_1-Score**	**Support**
Random Forest	**98.73%**	Relaxed (0)	0.99	0.99	0.99	705
		Stressed (1)	0.98	0.98	0.98	396
1D-CNN (PyTorch)	**96.37%**	Relaxed (0)	0.97	0.98	0.97	705
		Stressed (1)	0.96	0.94	0.95	396

5.2 Error Analysis and Confusion Matrix Breakdown

An analysis of the confusion matrix demonstrates balanced classification profiles:

- **Random Forest Baseline:** Out of 1,101 samples, the model produced only 7 False Positives (Relaxed classified as Stressed) and 7 False Negatives (Stressed classified as Relaxed), yielding a macro-averaged F_1-score of 0.99.
- **1D-CNN (PyTorch):** Produced 16 False Positives and 24 False Negatives. The convergence history indicated optimal validation loss at Epoch 20 ($\mathcal{L}_{\text{val}} = 0.1113$). Beyond Epoch 20, minor validation degradation was observed, confirming the suitability of early stopping.

6 Technical Challenges and Mitigation Strategies

6.1 Input Tensor Dimensionality Mismatch

PyTorch 1D convolutions enforce an input tensor dimensionality of:

$$(N, C_{\text{in}}, L_{\text{in}}) \tag{3}$$

where N is batch size, C_{in} is input channels, and L_{in} is sequence length. Standard tabular feature extraction yields 2D matrices $(N, 8)$. To resolve this, tensors were reshaped via `.unsqueeze(1)` to cast the single-window statistical metrics into a valid format $(N, 1, 8)$, enabling convolutional operations without requiring temporal zero-padding.

6.2 Mitigating Sensor Drift and Baseline Heterogeneity

Raw Electrodermal Activity displays natural tonic baseline shifts across different subjects. Standardizing features using the training set distribution ensured that large absolute sweat conductance values did not dominate smaller-amplitude cardiac standard deviation metrics.

7 Roadmap for Phase 4 (Evaluation and Bias Auditing)

While Phase 3 demonstrates high classification performance under a randomized 80/20 train/test split, this configuration presents a known vulnerability: **Subject Autocorrelation Bias**. When windows from the same subject appear in both training and test sets, the model can memorize idiosyncratic physiological baselines.

For Phase 4, the evaluation pipeline will be expanded to include:

1. **Subject-Independent Partitioning:** Training on subjects S2–S13 and testing strictly on unseen subjects S14–S17.
2. **Leave-One-Subject-Out (LOSO) Cross-Validation:** Systematically iterating through all 15 active subjects to quantify generalizability across unseen human bodies.
3. **Discrimination Metrics:** Computation of Receiver Operating Characteristic (ROC-AUC) curves and precision-recall trade-offs.

8 Conclusion

The Phase 3 milestones for Track A have been completed. Both the Random Forest classifier and the PyTorch 1D-CNN have been constructed, validated, and benchmarked on the WESAD dataset. The resulting scripts, execution outputs, and confusion matrix artifacts confirm operational models ready for Phase 4 evaluation.